

Chapter 1

File Handling & Serial Ports

ENEL712 Embedded Systems Design — Lecture 22

Summary

C# file handling via System.IO streams (text and binary), Windows file dialogs, directory/path operations, exception handling, and SerialPort communication including endianness considerations.

Topics Covered

- File types and streams
- C# classes for file handling
- Windows dialog boxes
- Directory and path operations
- Exception handling in file operations
- Serial port communication
- Endianness in multi-byte data

1.1 File Types and Streams

- C# treats files as sequential streams of bytes with a current position, length, and access mode.
- Text files: ASCII or Unicode (Notepad-style readable).
- Binary files: 0s and 1s, e.g., .bmp, .wav, executables — interpretable only with the data format.
- Files end with EOF or after a specified byte count.

1.2 C# Classes for File Handling

Class	Purpose	Key details
StreamReader	Read general text	Character input in specific encodings (e.g., ASCII).
StreamWriter	Write general text	Auto-handles encoding; not for binary data.
FileStream	Byte-level read/write	Requires a FileMode (Append, Create, Open) to open or create.
BinaryReader	Read binary data	Reads 8–64 bit primitives; needs a known data format.
BinaryWriter	Write binary data	Used with a Stream for structured formats (JPEG, WAV).

1.3 Windows Dialog Boxes

- OpenFileDialog: lets users select a file; non-visual element placed in the designer tray.
- SaveFileDialog: provides a save path and warns about existing files; doesn't create the file itself.
- Filter property restricts selectable types (e.g., 'MP3 Files | *.mp3').
- Best practice: check ShowDialog() == DialogResult.OK before proceeding.

1.4 Directory and Path Operations

- File methods: Copy, Create, Delete, Exists, Move, Open, Replace.
- Directory methods: CreateDirectory, Delete, Exists, Move.
- Path class: GetExtension, GetFileName, ChangeExtension.

1.5 Exception Handling in File Operations

Files can be read-only, missing, or in use — wrap operations in try/catch to handle exceptions and prevent crashes.

1.6 Serial Port Communication

- SerialPort object simplifies COM port communication; create graphically or programmatically (preferred).
- Methods are blocking by default; use timeouts or the DataReceived event for responsiveness.
- Key methods: Open(), GetPortNames() (static), ReadExisting() (drains buffer to string), Write() (bytes/strings).
- Considerations: buffering (multiple Reads may be needed), keep buffer cleared by reading even unwanted bytes.

1.7 Endianness in Multi-Byte Data

- Endianness is the order of bytes for multi-byte values.
- Little-Endian: LSB first — Intel/Windows PCs.
- Big-Endian: MSB first — network protocols and AVR.
- When PC ↔ AVR, the programmer must reverse multi-byte ordering before transmission.

1.8 Use Cases & Applications

- Logging serial data from an embedded device into CSV/binary files for offline analysis.
- Implementing simple firmware updaters that read a .hex/.bin file and stream it over UART.
- Building flight-data recorders / lab instruments where event-driven serial reception drives a live plot and disk log.
- Writing application configuration to disk on close and reading it back on launch.

1.9 Worked Example: Open + Read a Text File

```
var dlg = new OpenFileDialog
{
    Filter = "Text files|*.txt|All files|*.*"
};
if (dlg.ShowDialog() == DialogResult.OK)
{
    using StreamReader r = new StreamReader(dlg.FileName);
    string contents = r.ReadToEnd();
    textBox.Text = contents;
}
```

The using statement guarantees the stream closes even if an exception is thrown inside the block.

1.10 Worked Example: Event-Driven Serial Reception

```
SerialPort sp = new SerialPort("COM3", 115200);
sp.DataReceived += (s, e) =>
{
    string chunk = sp.ReadExisting();
    Invoke((Action)(() => logTextBox.AppendText(chunk)));
};
sp.Open();
```

Reading inside DataReceived avoids a busy loop. Marshalling back to the UI thread with Invoke is required because DataReceived runs on a worker thread.

1.11 Endianness Conversion

If a Big-Endian AVR sends a 32-bit value as bytes [b3, b2, b1, b0], a Little-Endian PC must reverse them before BitConverter.ToInt32:

```
byte[] frame = new byte[4]; // received from serial in Big-Endian order
if (BitConverter.IsLittleEndian) Array.Reverse(frame);
int value = BitConverter.ToInt32(frame, 0);
```

1.12 FileMode Decision Table

FileMode	Behaviour
Open	Open existing file; throw if missing
OpenOrCreate	Open existing file; create if missing
Create	Always create a new file (truncates existing)
CreateNew	Create a new file; throw if it already exists
Append	Open or create; seek to end so writes append
Truncate	Open existing file and discard its contents

Key Definitions

Stream

Abstraction treating I/O as a sequential byte flow with position/length/mode.

System.IO

C# namespace containing the file/directory classes.

Binary File

File of raw bytes interpretable only with a known data format.

EOF

End-of-File marker or specified byte count terminating a stream.

FileMode

Enum for FileStream open behaviour (Open / Create / Append).

Blocking Behavior

Default serial port behaviour halting execution until I/O completes.

Endianness

Byte ordering of multi-byte data — Little-Endian or Big-Endian.

Little-Endian

Least significant byte stored first (Intel).

Big-Endian

Most significant byte stored first (AVR / network protocols).

using Statement

Block that automatically calls Dispose() on the variable at scope exit, releasing files/sockets/etc.

DataReceived Event

SerialPort event fired on a worker thread when bytes arrive; preferred over polling.

IsLittleEndian

BitConverter property indicating the host's byte order; critical for safe wire-format conversions.

Asynchronous I/O

Pattern (async/await in C#) where I/O calls return a Task and don't block the calling thread.

Sources

- Lecture 22.pdf